

SANGOLA MAHAVIDYALAYA, SANGOLA

Department of Computer Science and Application

Subject: .NET Core | Practical Assignment - 03 | Date: 24/09/2025

PART A: Generic Collections (Q1 - Q25)

Q1. Sort a list of integers using List<T>.Sort().

```
using System;
using System.Collections.Generic;
class Q1 {
    static void Main() {
        List<int> nums = new List<int> { 5, 2, 8, 1, 9, 3, 7 };
        Console.WriteLine("Before: " + string.Join(", ", nums));
        nums.Sort();
        Console.WriteLine("After Sort: " + string.Join(", ", nums));
        nums.Sort((a, b) => b.CompareTo(a)); // descending
        Console.WriteLine("Descending: " + string.Join(", ", nums));
    }
}
```

// Output: Before: 5,2,8,1,9,3,7 | After: 1,2,3,5,7,8,9 | Desc: 9,8,7,5,3,2,1

Q2. Reverse a list of strings.

```
using System;
using System.Collections.Generic;
class Q2 {
    static void Main() {
        List<string> fruits = new List<string> { "Apple", "Banana", "Cherry", "Date", "Elderberry" };
        Console.WriteLine("Original: " + string.Join(", ", fruits));
        fruits.Reverse();
        Console.WriteLine("Reversed: " + string.Join(", ", fruits));
    }
}
```

Q3. Merge two lists and remove duplicates using HashSet<T>.

```
using System;
using System.Collections.Generic;
class Q3 {
    static void Main() {
        List<int> list1 = new List<int> { 1, 2, 3, 4, 5 };
        List<int> list2 = new List<int> { 4, 5, 6, 7, 8 };
        HashSet<int> merged = new HashSet<int>(list1);
        merged.UnionWith(list2); // adds list2, auto-removes duplicates
        Console.WriteLine("List1: " + string.Join(", ", list1));
        Console.WriteLine("List2: " + string.Join(", ", list2));
        Console.WriteLine("Merged (no duplicates): " + string.Join(", ", merged));
    }
}
```

// Output: Merged: 1,2,3,4,5,6,7,8

Q4. Find frequency of words using Dictionary<string,int>.

```
using System;
using System.Collections.Generic;
class Q4 {
    static void Main() {
        string sentence = "apple banana apple cherry banana apple date";
        string[] words = sentence.Split(' ');
```

```

Dictionary<string, int> freq = new Dictionary<string, int>();
foreach (string w in words) {
    if (freq.ContainsKey(w)) freq[w]++;
    else freq[w] = 1;
}
Console.WriteLine("Word Frequencies:");
foreach (var kv in freq)
    Console.WriteLine($" {kv.Key}: {kv.Value}");
}
}

```

Q5. Student record system using List<Student>.

```

using System;
using System.Collections.Generic;
class Student {
    public int RollNo; public string Name; public double Marks;
    public Student(int r, string n, double m) { RollNo=r; Name=n; Marks=m; }
    public void Display() => Console.WriteLine($"Roll:{RollNo} | {Name} | Marks:{Marks}");
}
class Q5 {
    static void Main() {
        List<Student> students = new List<Student>();
        students.Add(new Student(1, "Alice", 88));
        students.Add(new Student(2, "Bob", 72));
        students.Add(new Student(3, "Carol", 95));
        Console.WriteLine("--- Student Records ---");
        students.ForEach(s => s.Display());
        // Search
        var found = students.Find(s => s.RollNo == 2);
        Console.WriteLine("
Search Roll 2: " + found?.Name);
        // Remove
        students.RemoveAll(s => s.Marks < 75);
        Console.WriteLine("
After removing marks < 75:");
        students.ForEach(s => s.Display());
    }
}

```

Q6. Search element in a list using Contains().

```
using System;
using System.Collections.Generic;
class Q6 {
static void Main() {
List<string> cities = new List<string> { "Pune", "Mumbai", "Nagpur", "Solapur", "Nashik" };
string search = "Nagpur";
Console.WriteLine("Cities: " + string.Join(", ", cities));
Console.WriteLine($"Contains '{search}': " + cities.Contains(search));
Console.WriteLine($"Contains 'Delhi': " + cities.Contains("Delhi"));
int idx = cities.IndexOf(search);
Console.WriteLine($"Index of '{search}': " + idx);
}
}
```

Q7. Remove duplicates using LINQ Distinct().

```
using System;
using System.Collections.Generic;
using System.Linq;
class Q7 {
static void Main() {
List<int> nums = new List<int> { 1,2,3,2,4,3,5,1,6,5 };
Console.WriteLine("Original: " + string.Join(", ", nums));
List<int> unique = nums.Distinct().ToList();
Console.WriteLine("After Distinct(): " + string.Join(", ", unique));
}
}

// Output: After Distinct(): 1,2,3,4,5,6
```

Q8. Stack operations (Push, Pop, Peek).

```
using System;
using System.Collections.Generic;
class Q8 {
static void Main() {
Stack<int> stack = new Stack<int>();
stack.Push(10); stack.Push(20); stack.Push(30); stack.Push(40);
Console.WriteLine("Stack after pushes: " + string.Join(", ", stack));
Console.WriteLine("Peek (top): " + stack.Peek());
Console.WriteLine("Pop: " + stack.Pop());
Console.WriteLine("Pop: " + stack.Pop());
Console.WriteLine("Stack now: " + string.Join(", ", stack));
Console.WriteLine("Count: " + stack.Count);
Console.WriteLine("Contains 10: " + stack.Contains(10));
}
}
```

Q9. Queue operations (Enqueue, Dequeue, Peek).

```
using System;
using System.Collections.Generic;
class Q9 {
static void Main() {
Queue<string> queue = new Queue<string>();
queue.Enqueue("Alice"); queue.Enqueue("Bob");
queue.Enqueue("Carol"); queue.Enqueue("Dave");
Console.WriteLine("Queue: " + string.Join(", ", queue));
Console.WriteLine("Peek (front): " + queue.Peek());
}
```

```

Console.WriteLine("Dequeue: " + queue.Dequeue());
Console.WriteLine("Dequeue: " + queue.Dequeue());
Console.WriteLine("Queue now: " + string.Join(", ", queue));
}
}

```

Q10. Simulate Undo/Redo using Stack<T>.

```

using System;
using System.Collections.Generic;
class Q10 {
    static void Main() {
        Stack<string> undoStack = new Stack<string>();
        Stack<string> redoStack = new Stack<string>();
        // Do actions
        void DoAction(string action) {
            undoStack.Push(action);
            redoStack.Clear();
            Console.WriteLine("Action: " + action);
        }
        void Undo() {
            if (undoStack.Count > 0) {
                string a = undoStack.Pop();
                redoStack.Push(a);
                Console.WriteLine("Undo: " + a);
            }
        }
        void Redo() {
            if (redoStack.Count > 0) {
                string a = redoStack.Pop();
                undoStack.Push(a);
                Console.WriteLine("Redo: " + a);
            }
        }
        DoAction("Type 'Hello'");
        DoAction("Bold Text");
        DoAction("Insert Image");
        Undo(); Undo();
        Redo();
    }
}

```

Q11. Simulate a printer queue using Queue<T>.

```
using System;
using System.Collections.Generic;
class Q11 {
    static void Main() {
        Queue<string> printerQueue = new Queue<string>();
        printerQueue.Enqueue("Document1.pdf");
        printerQueue.Enqueue("Photo.jpg");
        printerQueue.Enqueue("Report.docx");
        printerQueue.Enqueue("Invoice.pdf");
        Console.WriteLine("Printer Queue: " + string.Join(", ", printerQueue));
        Console.WriteLine("
        Printing documents:");
        while (printerQueue.Count > 0) {
            string doc = printerQueue.Dequeue();
            Console.WriteLine($" Printing: {doc}");
        }
        Console.WriteLine("All documents printed!");
    }
}
```

Q12. Store key-value pairs using Dictionary<TKey,TValue>.

```
using System;
using System.Collections.Generic;
class Q12 {
    static void Main() {
        Dictionary<int, string> empDir = new Dictionary<int, string>();
        empDir.Add(101, "Alice");
        empDir.Add(102, "Bob");
        empDir[103] = "Carol"; // another way to add
        Console.WriteLine("All Employees:");
        foreach (var kv in empDir)
            Console.WriteLine($" ID:{kv.Key} -> {kv.Value}");
        // Access
        Console.WriteLine("
        ID 102: " + empDir[102]);
        // Check key
        Console.WriteLine("Has ID 104: " + empDir.ContainsKey(104));
        // Remove
        empDir.Remove(102);
        Console.WriteLine("After removing 102, Count: " + empDir.Count);
    }
}
```

Q13. Iterate over a dictionary and print keys and values.

```
using System;
using System.Collections.Generic;
class Q13 {
    static void Main() {
        Dictionary<string, int> scores = new Dictionary<string, int> {
            {"Alice", 90}, {"Bob", 75}, {"Carol", 88}, {"Dave", 65}
        };
        Console.WriteLine("Keys only:");
        foreach (string k in scores.Keys) Console.Write(k + " ");
        Console.WriteLine("
        Values only:");
    }
}
```

```

foreach (int v in scores.Values) Console.Write(v + " ");
Console.WriteLine("
Key-Value pairs:");
foreach (KeyValuePair<string,int> kv in scores)
Console.WriteLine($" {kv.Key} scored {kv.Value}");
}
}

```

Q14. Sort a dictionary by keys.

```

using System;
using System.Collections.Generic;
using System.Linq;
class Q14 {
static void Main() {
Dictionary<string, int> dict = new Dictionary<string, int> {
{"Zebra",3}, {"Apple",1}, {"Mango",2}, {"Banana",4}
};
var sortedByKey = dict.OrderBy(kv => kv.Key);
Console.WriteLine("Sorted by Key (A-Z):");
foreach (var kv in sortedByKey)
Console.WriteLine($" {kv.Key}: {kv.Value}");
}
}

```

Q15. Sort a dictionary by values.

```

using System;
using System.Collections.Generic;
using System.Linq;
class Q15 {
static void Main() {
Dictionary<string, int> marks = new Dictionary<string, int> {
{"Alice",88}, {"Bob",72}, {"Carol",95}, {"Dave",60}
};
var sortedByVal = marks.OrderBy(kv => kv.Value);
Console.WriteLine("Sorted by Value (ascending):");
foreach (var kv in sortedByVal)
Console.WriteLine($" {kv.Key}: {kv.Value}");
var sortedDesc = marks.OrderByDescending(kv => kv.Value);
Console.WriteLine("Sorted by Value (descending):");
foreach (var kv in sortedDesc)
Console.WriteLine($" {kv.Key}: {kv.Value}");
}
}

```

Q16. Subset and superset relationships using HashSet<T>.

```
using System;
using System.Collections.Generic;
class Q16 {
    static void Main() {
        HashSet<int> setA = new HashSet<int> { 1, 2, 3 };
        HashSet<int> setB = new HashSet<int> { 1, 2, 3, 4, 5 };
        HashSet<int> setC = new HashSet<int> { 6, 7 };
        Console.WriteLine("A is subset of B: " + setA.IsSubsetOf(setB));
        Console.WriteLine("B is superset of A: " + setB.IsSupersetOf(setA));
        Console.WriteLine("A is subset of C: " + setA.IsSubsetOf(setC));
        Console.WriteLine("A overlaps B: " + setA.Overlaps(setB));
        Console.WriteLine("A equals B: " + setA.SetEquals(setB));
    }
}
```

Q17. Intersection of two sets using IntersectWith().

```
using System;
using System.Collections.Generic;
class Q17 {
    static void Main() {
        HashSet<int> set1 = new HashSet<int> { 1,2,3,4,5 };
        HashSet<int> set2 = new HashSet<int> { 3,4,5,6,7 };
        Console.WriteLine("Set1: " + string.Join(",", set1));
        Console.WriteLine("Set2: " + string.Join(",", set2));
        HashSet<int> inter = new HashSet<int>(set1);
        inter.IntersectWith(set2);
        Console.WriteLine("Intersection: " + string.Join(",", inter));
    }
}
```

// Output: Intersection: 3,4,5

Q18. Union of two sets using UnionWith().

```
using System;
using System.Collections.Generic;
class Q18 {
    static void Main() {
        HashSet<int> set1 = new HashSet<int> { 1,2,3,4 };
        HashSet<int> set2 = new HashSet<int> { 3,4,5,6 };
        HashSet<int> union = new HashSet<int>(set1);
        union.UnionWith(set2);
        Console.WriteLine("Set1: " + string.Join(",", set1));
        Console.WriteLine("Set2: " + string.Join(",", set2));
        Console.WriteLine("Union: " + string.Join(",", union));
    }
}
```

// Output: Union: 1,2,3,4,5,6

Q19. Difference between two sets using ExceptWith().

```
using System;
using System.Collections.Generic;
class Q19 {
    static void Main() {
        HashSet<int> set1 = new HashSet<int> { 1,2,3,4,5 };
        HashSet<int> set2 = new HashSet<int> { 3,4,5,6,7 };
        HashSet<int> diff = new HashSet<int>(set1);
        diff.ExceptWith(set2); // set1 - set2
    }
}
```

```
Console.WriteLine("Set1: " + string.Join(",", set1));
Console.WriteLine("Set2: " + string.Join(",", set2));
Console.WriteLine("Set1-Set2: " + string.Join(",", diff));
}
}
```

// Output: Difference: 1,2

Q20. Linked list operations using LinkedList<T>.

```
using System;
using System.Collections.Generic;
class Q20 {
static void Main() {
LinkedList<int> ll = new LinkedList<int>();
ll.AddLast(10); ll.AddLast(20); ll.AddLast(30);
ll.AddFirst(5); // add at front
ll.AddAfter(ll.Find(20), 25); // add after 20
Console.Write("LinkedList: ");
foreach (int n in ll) Console.Write(n + " ");
Console.WriteLine();
ll.Remove(10); // remove by value
Console.Write("After Remove(10): ");
foreach (int n in ll) Console.Write(n + " ");
Console.WriteLine();
Console.WriteLine("Count: " + ll.Count);
Console.WriteLine("First: " + ll.First.Value + " | Last: " + ll.Last.Value);
}
}
```


Q21. Find duplicates in an array using Dictionary<int,int>.

```
using System;
using System.Collections.Generic;
class Q21 {
static void Main() {
int[] arr = { 1, 2, 3, 2, 4, 3, 5, 1, 6 };
Dictionary<int, int> freq = new Dictionary<int, int>();
foreach (int n in arr) {
if (freq.ContainsKey(n)) freq[n]++;
else freq[n] = 1;
}
Console.WriteLine("Duplicates found:");
foreach (var kv in freq)
if (kv.Value > 1)
Console.WriteLine($" {kv.Key} appears {kv.Value} times");
}
}
```

// Output: 2 appears 2 times, 3 appears 2 times, 1 appears 2 times

Q22. Group employees by department using Dictionary<string,List<Employee>>.

```
using System;
using System.Collections.Generic;
class Employee {
public string Name, Dept;
public Employee(string n, string d) { Name=n; Dept=d; }
}
class Q22 {
static void Main() {
List<Employee> emps = new List<Employee> {
new Employee("Alice","IT"), new Employee("Bob","HR"),
new Employee("Carol","IT"), new Employee("Dave","Finance"),
new Employee("Eve","HR"), new Employee("Frank","IT")
};
Dictionary<string, List<Employee>> groups = new Dictionary<string, List<Employee>>();
foreach (var e in emps) {
if (!groups.ContainsKey(e.Dept))
groups[e.Dept] = new List<Employee>();
groups[e.Dept].Add(e);
}
foreach (var kv in groups) {
Console.Write($"Dept [{kv.Key}]: ");
kv.Value.ForEach(e => Console.Write(e.Name + " "));
Console.WriteLine();
}
}
}
```

Q23. Count vowels in a string using Dictionary<char,int>.

```
using System;
using System.Collections.Generic;
class Q23 {
static void Main() {
string text = "Hello World from DotNet";
char[] vowels = {'a','e','i','o','u','A','E','I','O','U'};
Dictionary<char, int> vowelCount = new Dictionary<char, int>();
foreach (char c in text) {
```

```

if (Array.IndexOf(vowels, c) >= 0) {
    char lower = char.ToLower(c);
    if (vowelCount.ContainsKey(lower)) vowelCount[lower]++;
    else vowelCount[lower] = 1;
}
}
Console.WriteLine($"Text: \"{text}\"");
Console.WriteLine("Vowel counts:");
foreach (var kv in vowelCount)
    Console.WriteLine($" '{kv.Key}': {kv.Value}");
int total = 0;
foreach (var v in vowelCount.Values) total += v;
Console.WriteLine("Total vowels: " + total);
}
}

```

Q24. Demonstrate SortedList<TKey,TValue>.

```

using System;
using System.Collections.Generic;
class Q24 {
    static void Main() {
        SortedList<string, int> sl = new SortedList<string, int>();
        sl.Add("Zebra", 5);
        sl.Add("Apple", 1);
        sl.Add("Mango", 3);
        sl.Add("Banana", 2);
        sl["Cherry"] = 4; // add via indexer
        Console.WriteLine("SortedList (auto-sorted by key):");
        foreach (var kv in sl)
            Console.WriteLine($" {kv.Key}: {kv.Value}");
        Console.WriteLine("Keys: " + string.Join(", ", sl.Keys));
        Console.WriteLine("Values: " + string.Join(", ", sl.Values));
        Console.WriteLine("Index of 'Mango': " + sl.IndexOfKey("Mango"));
        sl.Remove("Apple");
        Console.WriteLine("After Remove 'Apple', Count: " + sl.Count);
    }
}

```

Q25. Phonebook using Dictionary<string,string>.

```

using System;
using System.Collections.Generic;
class Q25 {
    static void Main() {
        Dictionary<string, string> phonebook = new Dictionary<string, string>();
        phonebook["Alice"] = "9876543210";
        phonebook["Bob"] = "8765432109";
        phonebook["Carol"] = "7654321098";
        phonebook["Dave"] = "6543210987";

        Console.WriteLine("--- Phonebook ---");
        foreach (var kv in phonebook)
            Console.WriteLine($" {kv.Key,-10}: {kv.Value}");

        // Search
        string search = "Bob";
        if (phonebook.TryGetValue(search, out string num))

```

```
Console.WriteLine($"
{search}'s number: {num}");
else
Console.WriteLine($"{search} not found.");

// Update
phonebook["Alice"] = "9999999999";
Console.WriteLine("Updated Alice: " + phonebook["Alice"]);

// Delete
phonebook.Remove("Dave");
Console.WriteLine("After Delete Dave, Count: " + phonebook.Count);
}
}
```

PART B: LINQ with Database (Q26 - Q29)

Note: The following programs use Entity Framework Core (EF Core) with SQL Server. In-memory collections are used to demonstrate LINQ logic clearly.

Q26. Employees + Departments + EmployeeProjects

Setup: Table Creation SQL + Sample Data

```
-- SQL Table Creation
CREATE TABLE Departments (
    DepartmentId INT PRIMARY KEY IDENTITY,
    DepartmentName NVARCHAR(100) NOT NULL
);
CREATE TABLE Employees (
    EmployeeId INT PRIMARY KEY IDENTITY,
    Name NVARCHAR(100) NOT NULL,
    DepartmentId INT NULL,
    ManagerId INT NULL,
    Salary DECIMAL(18,2) NOT NULL,
    HireDate DATE NOT NULL,
    FOREIGN KEY (DepartmentId) REFERENCES Departments(DepartmentId),
    FOREIGN KEY (ManagerId) REFERENCES Employees(EmployeeId)
);
CREATE TABLE EmployeeProjects (
    EmployeeId INT NOT NULL,
    ProjectId INT NOT NULL,
    PRIMARY KEY (EmployeeId, ProjectId)
);

-- Sample Data
INSERT INTO Departments VALUES ('IT'),('HR'),('Finance');
INSERT INTO Employees VALUES
('Alice',1,NULL,70000,'2019-03-01'),
('Bob', 1,1, 55000,'2021-06-15'),
('Carol',2,NULL,48000,'2018-11-10'),
('Dave', 2,3, 35000,'2022-01-20'),
('Eve', 3,NULL,80000,'2017-05-05'),
('Frank',1,1, 60000,'2023-08-01'),
('Anna', 3,5, 52000,'2020-04-12');
INSERT INTO EmployeeProjects VALUES (1,101),(2,101),(3,102),(5,103),(6,104);

// C# Model Classes + LINQ Queries (using in-memory data)
using System;
using System.Collections.Generic;
using System.Linq;

class Department { public int DeptId; public string Name; }
class Emp {
    public int Id, DeptId; public int? MgrId;
    public string Name; public decimal Salary; public DateTime HireDate;
}
class EmpProject { public int EmpId, ProjId; }

class Q26 {
    static void Main() {
        var depts = new List<Department> {
            new Department{DeptId=1,Name="IT"},
            new Department{DeptId=2,Name="HR"},
            new Department{DeptId=3,Name="Finance"}
```

```

};
var emps = new List<Emp> {
    new Emp{Id=1,Name="Alice",DeptId=1,MgrId=null, Salary=70000,HireDate=new DateTime(2019,3,1)},
    new Emp{Id=2,Name="Bob", DeptId=1,MgrId=1, Salary=55000,HireDate=new DateTime(2021,6,15)},
    new Emp{Id=3,Name="Carol",DeptId=2,MgrId=null, Salary=48000,HireDate=new DateTime(2018,11,10)},
    new Emp{Id=4,Name="Dave", DeptId=2,MgrId=3, Salary=35000,HireDate=new DateTime(2022,1,20)},
    new Emp{Id=5,Name="Eve", DeptId=3,MgrId=null, Salary=80000,HireDate=new DateTime(2017,5,5)},
    new Emp{Id=6,Name="Frank",DeptId=1,MgrId=1, Salary=60000,HireDate=new DateTime(2023,8,1)},
    new Emp{Id=7,Name="Anna", DeptId=3,MgrId=5, Salary=52000,HireDate=new DateTime(2020,4,12)}
};

var ep = new List<EmpProject> {
    new EmpProject{EmpId=1,ProjId=101}, new EmpProject{EmpId=2,ProjId=101},
    new EmpProject{EmpId=3,ProjId=102}, new EmpProject{EmpId=5,ProjId=103},
    new EmpProject{EmpId=6,ProjId=104}
};

// Q26.1 Join Employees with Departments
Console.WriteLine("1. Employee + Department:");
var q1 = from e in emps
join d in depts on e.DeptId equals d.DeptId
select new { e.Name, Dept = d.Name };
foreach (var r in q1) Console.WriteLine($" {r.Name} - {r.Dept}");

// Q26.2 Employees earning more than 50000 per dept
Console.WriteLine("2. Salary > 50000 per dept:");
var q2 = emps.Where(e => e.Salary > 50000)
.Join(depts, e => e.DeptId, d => d.DeptId,
(e,d) => new{e.Name,d.Name,e.Salary});
foreach (var r in q2) Console.WriteLine($" {r.Name}({r.Name}) Rs.{r.Salary}");

// Q26.3 Count employees per dept
Console.WriteLine("3. Count per dept:");
var q3 = emps.GroupBy(e => e.DeptId)
.Join(depts, g => g.Key, d => d.DeptId,
(g,d) => new{Dept=d.Name,Count=g.Count()});
foreach (var r in q3) Console.WriteLine($" {r.Dept}: {r.Count}");

// Q26.4 Max salary per dept
Console.WriteLine("4. Max salary per dept:");
var q4 = emps.GroupBy(e => e.DeptId)
.Join(depts, g => g.Key, d => d.DeptId,
(g,d) => new{Dept=d.Name,Max=g.Max(e=>e.Salary)});
foreach (var r in q4) Console.WriteLine($" {r.Dept}: Rs.{r.Max}");

// Q26.5 Employees without manager
Console.WriteLine("5. No Manager:");
emps.Where(e => e.MgrId == null)
.ToList().ForEach(e => Console.WriteLine(" " + e.Name));

// Q26.6 Group by dept, order by salary
Console.WriteLine("6. Group by dept, order by salary:");
var q6 = emps.OrderBy(e=>e.Salary)
.Join(depts,e=>e.DeptId,d=>d.DeptId,(e,d)=>new{e.Name,d.Name,e.Salary})
.GroupBy(x=>x.Name);
foreach(var g in q6){ Console.WriteLine($" Dept:{g.Key}");
foreach(var r in g) Console.WriteLine($" {r.Name} Rs.{r.Salary}"); }

```

```

// Q26.7 Hired after 2020
Console.WriteLine("7. Hired after 2020:");
emps.Where(e => e.HireDate.Year > 2020)
.ToList().ForEach(e => Console.WriteLine($" {e.Name} ({e.HireDate:yyyy-MM-dd})"));

// Q26.8 Name starts with 'A'
Console.WriteLine("8. Name starts with A:");
emps.Where(e => e.Name.StartsWith("A"))
.ToList().ForEach(e => Console.WriteLine(" " + e.Name));

// Q26.9 Above dept average salary
Console.WriteLine("9. Above dept avg salary:");
var avgByDept = emps.GroupBy(e => e.DeptId)
.ToDictionary(g => g.Key, g => g.Average(e => e.Salary));
emps.Where(e => e.Salary > avgByDept[e.DeptId])
.Join(depts, e=>e.DeptId, d=>d.DeptId,
(e,d) => new{e.Name, d.Name, e.Salary, Avg=avgByDept[e.DeptId]})
.ToList().ForEach(r =>
Console.WriteLine($" {r.Name} Rs.{r.Salary} (Avg Rs.{r.Avg:F0})"));

// Q26.10 Employees on projects
Console.WriteLine("10. Employees on projects:");
var q10 = emps.Join(ep, e=>e.Id, p=>p.EmpId, (e,p) => new{e.Name, p.ProjId});
foreach(var r in q10) Console.WriteLine($" {r.Name} - Project {r.ProjId}");
}
}

```

Q27. Students + Courses + StudentCourses

```
-- SQL Table Creation
CREATE TABLE Students (
StudentId INT PRIMARY KEY IDENTITY,
Name NVARCHAR(100) NOT NULL,
Marks INT NOT NULL,
Year INT NOT NULL
);
CREATE TABLE Courses (
CourseId INT PRIMARY KEY IDENTITY,
CourseName NVARCHAR(100) NOT NULL,
Credits INT NOT NULL
);
CREATE TABLE StudentCourses (
StudentId INT NOT NULL,
CourseId INT NOT NULL,
PRIMARY KEY (StudentId, CourseId)
);

-- Sample Data
INSERT INTO Students VALUES
('Alice',85,2),('Bob',72,3),('Carol',91,1),
('Dave',45,2),('Eve',78,3),('Frank',38,1);
INSERT INTO Courses VALUES
('Mathematics',4),('Physics',3),('Chemistry',3),('Computer Science',4);
INSERT INTO StudentCourses VALUES
(1,1),(1,4),(2,1),(2,2),(2,4),(3,3),(3,4),(4,2),(5,1),(5,3),(6,4);

using System;
using System.Collections.Generic;
using System.Linq;

class Stud { public int Id, Marks, Year; public string Name; }
class Course { public int Id, Credits; public string Name; }
class StCourse { public int SId, CId; }

class Q27 {
static void Main() {
var studs = new List<Stud> {
new Stud{Id=1,Name="Alice",Marks=85,Year=2},
new Stud{Id=2,Name="Bob", Marks=72,Year=3},
new Stud{Id=3,Name="Carol",Marks=91,Year=1},
new Stud{Id=4,Name="Dave", Marks=45,Year=2},
new Stud{Id=5,Name="Eve", Marks=78,Year=3},
new Stud{Id=6,Name="Frank",Marks=38,Year=1}
};
var courses = new List<Course> {
new Course{Id=1,Name="Maths", Credits=4},
new Course{Id=2,Name="Physics", Credits=3},
new Course{Id=3,Name="Chemistry",Credits=3},
new Course{Id=4,Name="CS", Credits=4}
};
var sc = new List<StCourse> {
new StCourse{SId=1,CId=1},new StCourse{SId=1,CId=4},
new StCourse{SId=2,CId=1},new StCourse{SId=2,CId=2},new StCourse{SId=2,CId=4},
new StCourse{SId=3,CId=3},new StCourse{SId=3,CId=4},
```

```

new StCourse{Sid=4,Cid=2},new StCourse{Sid=5,Cid=1},
new StCourse{Sid=5,Cid=3},new StCourse{Sid=6,Cid=4}
};

// 27.1 Join students with courses
Console.WriteLine("1. Students and their Courses:");
var q1 = from x in sc
join s in studs on x.Sid equals s.Id
join c in courses on x.Cid equals c.Id
select new {s.Name, Course=c.Name};
foreach(var r in q1) Console.WriteLine($" {r.Name} -> {r.Course}");

// 27.2 Marks > 80
Console.WriteLine("2. Marks > 80:");
studs.Where(s => s.Marks > 80)
.ToList().ForEach(s => Console.WriteLine($" {s.Name}: {s.Marks}"));

// 27.3 Courses per student
Console.WriteLine("3. Courses per student:");
var q3 = sc.GroupBy(x => x.Sid)
.Join(studs, g=>g.Key, s=>s.Id,
(g,s) => new{s.Name,
Courses=string.Join(", ",g.Select(x=>courses.First(c=>c.Id==x.Cid).Name))});
foreach(var r in q3) Console.WriteLine($" {r.Name}: {r.Courses}");

// 27.4 Count courses per student
Console.WriteLine("4. Course count per student:");
sc.GroupBy(x => x.Sid)
.Join(studs,g=>g.Key,s=>s.Id,(g,s)=>new{s.Name,Count=g.Count()})
.ToList().ForEach(r => Console.WriteLine($" {r.Name}: {r.Count} courses"));

// 27.5 Most popular course
Console.WriteLine("5. Most popular course:");
var popular = sc.GroupBy(x => x.Cid)
.OrderByDescending(g => g.Count())
.First();
Console.WriteLine($" {courses.First(c=>c.Id==popular.Key).Name} ({popular.Count()} students)");

// 27.6 Students in more than 1 course
Console.WriteLine("6. Students in >1 course:");
sc.GroupBy(x=>x.Sid).Where(g=>g.Count(>1)
.Join(studs,g=>g.Key,s=>s.Id,(g,s)=>s.Name)
.ToList().ForEach(n => Console.WriteLine(" " + n));

// 27.7 Sort by marks
Console.WriteLine("7. Sort by Marks:");
studs.OrderByDescending(s=>s.Marks)
.ToList().ForEach(s=>Console.WriteLine($" {s.Name}: {s.Marks}"));

// 27.8 Average marks
Console.WriteLine("8. Average Marks: " + studs.Average(s => s.Marks).ToString("F2"));

// 27.9 All passed (marks > 40)
Console.WriteLine("9. All students passed: " + studs.All(s => s.Marks > 40));

// 27.10 Skip 2 take 3

```



```
Console.WriteLine("10. Skip 2, Take 3:");  
studs.Skip(2).Take(3).ToList().ForEach(s=>Console.WriteLine($" {s.Name}"));  
}  
}
```

Q28. Customers + Orders + OrderDetails + Products

```
-- SQL Table Creation
CREATE TABLE Customers (
CustomerId INT PRIMARY KEY IDENTITY,
Name NVARCHAR(100) NOT NULL
);
CREATE TABLE Orders (
OrderId INT PRIMARY KEY IDENTITY,
CustomerId INT NOT NULL,
OrderDate DATE NOT NULL,
TotalAmount DECIMAL(18,2) NOT NULL
);
CREATE TABLE Products (
ProductId INT PRIMARY KEY IDENTITY,
ProductName NVARCHAR(100) NOT NULL,
Price DECIMAL(18,2) NOT NULL
);
CREATE TABLE OrderDetails (
OrderId INT NOT NULL,
ProductId INT NOT NULL,
Quantity INT NOT NULL,
PRIMARY KEY (OrderId, ProductId)
);

-- Sample Data
INSERT INTO Customers VALUES ('Alice'),('Bob'),('Carol'),('Dave');
INSERT INTO Orders VALUES
(1, '2022-01-10', 5000), (1, '2024-03-05', 8000),
(2, '2023-07-15', 3500), (3, '2024-01-20', 12000), (4, '2021-11-30', 2000);
INSERT INTO Products VALUES
('Laptop', 45000), ('Phone', 20000), ('Tablet', 15000), ('Headphone', 2000);
INSERT INTO OrderDetails VALUES
(1, 1, 1), (1, 4, 2), (2, 2, 1), (3, 3, 1), (4, 1, 1), (4, 2, 2), (5, 4, 3);

using System;
using System.Collections.Generic;
using System.Linq;

class Cust { public int Id; public string Name; }
class Order2 { public int Id, CustId; public DateTime Date; public decimal Total; }
class Prod { public int Id; public string Name; public decimal Price; }
class OD { public int OrdId, ProdId, Qty; }

class Q28 {
static void Main() {
var custs = new List<Cust> {
new Cust{Id=1,Name="Alice"}, new Cust{Id=2,Name="Bob"},
new Cust{Id=3,Name="Carol"}, new Cust{Id=4,Name="Dave"}
};
var orders = new List<Order2> {
new Order2{Id=1,CustId=1,Date=new DateTime(2022,1,10),Total=5000},
new Order2{Id=2,CustId=1,Date=new DateTime(2024,3,5), Total=8000},
new Order2{Id=3,CustId=2,Date=new DateTime(2023,7,15),Total=3500},
new Order2{Id=4,CustId=3,Date=new DateTime(2024,1,20),Total=12000},
new Order2{Id=5,CustId=4,Date=new DateTime(2021,11,30),Total=2000}
};
}
```

```

var prods = new List<Prod> {
    new Prod{Id=1,Name="Laptop",Price=45000}, new Prod{Id=2,Name="Phone",Price=20000},
    new Prod{Id=3,Name="Tablet",Price=15000}, new Prod{Id=4,Name="Headphone",Price=2000}
};

var od = new List<OD> {
    new OD{OrdId=1,ProdId=1,Qty=1}, new OD{OrdId=1,ProdId=4,Qty=2},
    new OD{OrdId=2,ProdId=2,Qty=1}, new OD{OrdId=3,ProdId=3,Qty=1},
    new OD{OrdId=4,ProdId=1,Qty=1}, new OD{OrdId=4,ProdId=2,Qty=2},
    new OD{OrdId=5,ProdId=4,Qty=3}
};

// 28.1 Join customers with orders
Console.WriteLine("1. Customers and Orders:");
orders.Join(custs,o=>o.CustId,c=>c.Id,(o,c)=>new{c.Name,o.Id,o.Total})
.ToList().ForEach(r=>Console.WriteLine($" {r.Name} - Order#{r.Id} Rs.{r.Total}"));

// 28.2 Total orders per customer
Console.WriteLine("2. Orders per customer:");
orders.GroupBy(o=>o.CustId)
.Join(custs,g=>g.Key,c=>c.Id,(g,c)=>new{c.Name,Count=g.Count()})
.ToList().ForEach(r=>Console.WriteLine($" {r.Name}: {r.Count}"));

// 28.3 Total purchase per customer
Console.WriteLine("3. Total purchase per customer:");
orders.GroupBy(o=>o.CustId)
.Join(custs,g=>g.Key,c=>c.Id,(g,c)=>new{c.Name>Total=g.Sum(o=>o.Total)})
.ToList().ForEach(r=>Console.WriteLine($" {r.Name}: Rs.{r.Total}"));

// 28.4 Join orders with products
Console.WriteLine("4. Orders with Products:");
od.Join(prods,d=>d.ProdId,p=>p.Id,(d,p)=>new{d.OrdId,p.Name,d.Qty})
.ToList().ForEach(r=>Console.WriteLine($" Order#{r.OrdId}: {r.Name} x{r.Qty}"));

// 28.5 Most ordered product
Console.WriteLine("5. Most ordered product:");
var top = od.GroupBy(d=>d.ProdId).OrderByDescending(g=>g.Sum(d=>d.Qty)).First();
Console.WriteLine($" {prods.First(p=>p.Id==top.Key).Name} (qty={top.Sum(d=>d.Qty)})");

// 28.6 Total qty per product
Console.WriteLine("6. Total qty per product:");
od.GroupBy(d=>d.ProdId)
.Join(prods,g=>g.Key,p=>p.Id,(g,p)=>new{p.Name,Qty=g.Sum(d=>d.Qty)})
.ToList().ForEach(r=>Console.WriteLine($" {r.Name}: {r.Qty}"));

// 28.7 Highest value order
Console.WriteLine("7. Highest order:");
var high = orders.OrderByDescending(o=>o.Total).First();
Console.WriteLine($" Order#{high.Id} Rs.{high.Total}");

// 28.8 Customers after 2023
Console.WriteLine("8. Customers after 2023:");
orders.Where(o=>o.Date.Year>2023)
.Join(custs,o=>o.CustId,c=>c.Id,(o,c)=>new{c.Name,o.Date})
.ToList().ForEach(r=>Console.WriteLine($" {r.Name} on {r.Date:yyyy-MM-dd}"));

// 28.9 Paging

```

```
Console.WriteLine("9. Paging (page 1, size 2):");
orders.Skip(0).Take(2).ToList().ForEach(o=>Console.WriteLine($" Order#{o.Id}"));

// 28.10 Products sorted by price desc
Console.WriteLine("10. Products by price (desc):");
prods.OrderByDescending(p=>p.Price)
.ToList().ForEach(p=>Console.WriteLine($" {p.Name}: Rs.{p.Price}"));
}
}
```

Q29. Employees + Projects + Departments

```
-- SQL Table Creation (Projects table - Employees & Departments from Q26)
CREATE TABLE Projects (
ProjectId INT PRIMARY KEY IDENTITY,
ProjectName NVARCHAR(100) NOT NULL,
DepartmentId INT NULL,
FOREIGN KEY (DepartmentId) REFERENCES Departments(DepartmentId)
);

-- Reuse EmployeeProjects from Q26

-- Sample Data
INSERT INTO Projects VALUES
('Web Portal',1),('HR System',2),('Finance App',3),('Mobile App',1);
INSERT INTO EmployeeProjects VALUES (1,1),(2,1),(3,2),(5,3),(6,4),(1,4);

using System;
using System.Collections.Generic;
using System.Linq;

class Proj { public int Id, DeptId; public string Name; }
// Reuse Emp, Department, EmpProject classes from Q26

class Q29 {
static void Main() {
// Reuse same data from Q26 + new Projects
var depts = new List<(int Id, string Name)> {(1,"IT"),(2,"HR"),(3,"Finance")};
var emps = new List<(int Id,string Name,int DeptId)> {
(1,"Alice",1),(2,"Bob",1),(3,"Carol",2),(4,"Dave",2),(5,"Eve",3),(6,"Frank",1)
};
var projs = new List<(int Id,string Name,int DeptId)> {
(1,"Web Portal",1),(2,"HR System",2),(3,"Finance App",3),(4,"Mobile App",1)
};
var ep = new List<(int EId,int PId)> {
(1,1),(2,1),(3,2),(5,3),(6,4),(1,4)
};

// 29.1 Join Employees, Projects, Departments
Console.WriteLine("1. Employee - Project - Department:");
var q1 = from x in ep
join e in emps on x.EId equals e.Id
join p in projs on x.PId equals p.Id
join d in depts on e.DeptId equals d.Id
select new{e.Name,Proj=p.Name,Dept=d.Name};
foreach(var r in q1) Console.WriteLine($" {r.Name} | {r.Proj} | {r.Dept}");

// 29.2 Projects per department
Console.WriteLine("2. Projects per dept:");
projs.Join(depts,p=>p.DeptId,d=>d.Id,(p,d)=>new{d.Name,Proj=p.Name})
.GroupBy(x=>x.Name)
.ToList().ForEach(g=>{ Console.Write($" {g.Key}: ");
g.ToList().ForEach(x=>Console.Write(x.Proj+" ")); Console.WriteLine(); });

// 29.3 Count projects per dept
Console.WriteLine("3. Project count per dept:");
projs.GroupBy(p=>p.DeptId)
.Join(depts,g=>g.Key,d=>d.Id,(g,d)=>new{d.Name,Count=g.Count()})
```

```

.ToList().ForEach(r=>Console.WriteLine($" {r.Name}: {r.Count}"));

// 29.4 Employees on multiple projects
Console.WriteLine("4. Employees on multiple projects:");
ep.GroupBy(x=>x.EId).Where(g=>g.Count(>1)
.Join(emps, g=>g.Key, e=>e.Id, (g, e)=>new{e.Name, Count=g.Count()}))
.ToList().ForEach(r=>Console.WriteLine($" {r.Name}: {r.Count} projects"));

// 29.5 Group employees by project
Console.WriteLine("5. Employees grouped by project:");
ep.GroupBy(x=>x.PId)
.ToList().ForEach(g=>{
var pname = projs.First(p=>p.Id==g.Key).Name;
Console.WriteLine($" {pname}: ");
g.ToList().ForEach(x=>Console.WriteLine(emps.First(e=>e.Id==x.EId).Name+" "));
Console.WriteLine(); });

// 29.6 Employees in IT department projects
Console.WriteLine("6. IT Dept employees on projects:");
var itEmps = emps.Where(e=>e.DeptId==1).Select(e=>e.Id).ToHashSet();
ep.Where(x=>itEmps.Contains(x.EId))
.Join(emps, x=>x.EId, e=>e.Id, (x, e)=>new{e.Name, ProjId=x.PId})
.Join(projs, x=>x.ProjId, p=>p.Id, (x, p)=>new{x.Name, Proj=p.Name})
.ToList().ForEach(r=>Console.WriteLine($" {r.Name} -> {r.Proj}"));

// 29.7 Sort employees by project name
Console.WriteLine("7. Employees sorted by project:");
ep.Join(emps, x=>x.EId, e=>e.Id, (x, e)=>new{e.Name, PId=x.PId})
.Join(projs, x=>x.PId, p=>p.Id, (x, p)=>new{x.Name, Proj=p.Name})
.OrderBy(r=>r.Proj)
.ToList().ForEach(r=>Console.WriteLine($" {r.Name} | {r.Proj}"));

// 29.8 Dept with most projects
Console.WriteLine("8. Dept with most projects:");
var topDept = projs.GroupBy(p=>p.DeptId).OrderByDescending(g=>g.Count()).First();
Console.WriteLine($" {depts.First(d=>d.Id==topDept.Key).Name} ({topDept.Count()} projects)");
}
}

```